Lab 4: Sobel Edge Detector – Hybrid Programming Model

Jarrell Reynolds & Tristan Barber — EEL6763 (Spring 2026)

PART 1 –

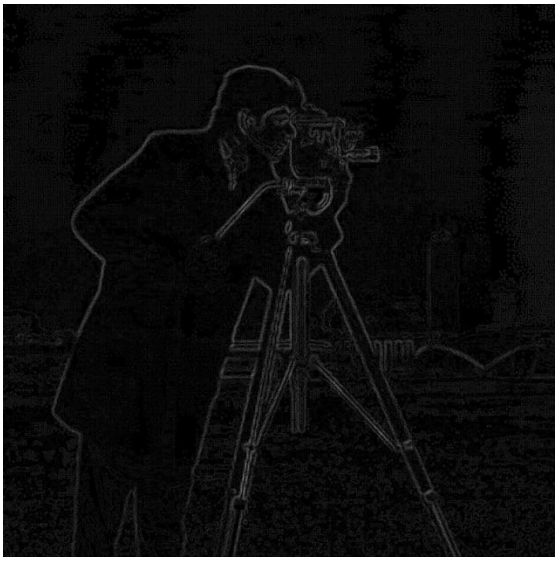
```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <time.h>

#define WIDTH 5000
#define HEIGHT 5000

uint8_t input[HEIGHT][WIDTH];
uint8_t output[HEIGHT][WIDTH];

/* ttype: type to use for representing time */
typedef double ttype;

/* Find the time difference. */
ttype tdiff(struct timespec a, struct timespec b) {
  ttype dt = (( b.tv_sec - a.tv_sec ) + ( b.tv_nsec - a.tv_nsec ) / 1E9);
  return dt;
}

/* Return the current time. */
```

```
struct timespec now() {
  struct timespec t;
  clock_gettime(CLOCK_REALTIME, &t);
  return t;
}

void read_image(const char *filename) {
   FILE *fp = fopen(filename, "r");

   if (!fp) {
      perror("Error opening input file");
      exit(1);
   }

   for (int r = 0; r < HEIGHT; r++) {
      for (int c = 0; c < WIDTH; c++) {
         int val;
         fscanf(fp, "%d", &val);
         input[r][c] = (uint8_t)val;
      }
   }

   fclose(fp);
}

void write_image(const char *filename) {
   FILE *fp = fopen(filename, "w");
```

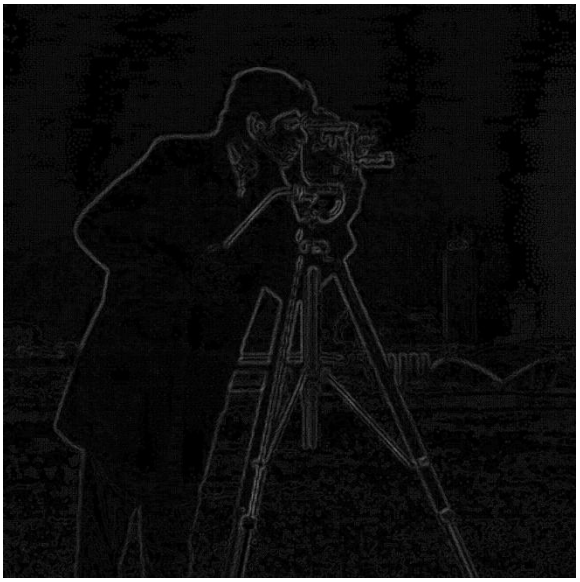
```
    if (!fp) {
        perror("Error opening output file");
        exit(1);
    }

    for (int r = 0; r < HEIGHT; r++) {
        for (int c = 0; c < WIDTH; c++) {
            fprintf(fp, "%d ", output[r][c]);
        }

        fprintf(fp, "\n");
    }

    fclose(fp);
}

void sobel() {
    // skip the outer borders like lab 3
    for (int r = 1; r < HEIGHT - 1; r++) {
        for (int c = 1; c < WIDTH - 1; c++) {
            int p1 = input[r-1][c-1];
            int p2 = input[r-1][c];
            int p3 = input[r-1][c+1];
            int q1 = input[r][c-1];
            int q2 = input[r][c];
            int q3 = input[r][c+1];
```

```
        int r1 = input[r+1][c-1];
        int r2 = input[r+1][c];
        int r3 = input[r+1][c+1];

        int qx = (p1 - r1) + 2*(p2 - r2) + (p3 - r3);
        int qy = (p1 - p3) + 2*(q1 - q3) + (r1 - r3);
        int magnitude = abs(qx) + abs(qy);

        // clamp results to 8-bit
        if (magnitude > 255)
          magnitude = 255;

        output[r][c] = (uint8_t)magnitude;
      }
    }
}

int main(int argc, char *argv[]) {
  if (argc != 2) {
    printf("Usage: %s input.txt\n", argv[0]);
    return 1;
  }

  struct timespec begin, end;

  read_image(argv[1]);
```

```
    begin = now();

    sobel();

    write_image("output.txt");

    end = now();

    printf("Sobel edge detection complete.\n");
    printf("total time: %.8f sec\n", tdiff(begin, end));

    return 0;
}
```



Part 2



Performance Study

Config	Nodes	Ranks	Threads	Schedule	Time (s)
Baseline	1	1	1	N/A	1.0332
1-node	1	4	4	static	1.0817
2-node	2	4	4	static	**1.0141**
4-node	4	4	4	static	1.1005
2-node	2	4	4	dynamic,16	1.1712
2-node	2	4	4	dynamic,64	2.0367

Good Configuration

The best-performing configuration was the 2-node setup using 4 MPI ranks and 4 OpenMP threads per rank with static scheduling. This configuration achieved the lowest runtime of 1.0141 seconds. It likely performed best due to a balance between parallel workload distribution and communication overhead. The workload in Sobel filtering is evenly distributed, making static scheduling efficient.

Bad Configuration

The worst-performing configuration was the 2-node run using dynamic scheduling with a chunk size of 64, which resulted in a runtime of 2.0367 seconds. This configuration performed poorly due to the overhead introduced by dynamic scheduling. Since the Sobel algorithm performs uniform computation across rows, dynamic scheduling adds unnecessary overhead without improving load balancing.

How Best Configuration Was Found

Multiple configurations were tested by varying the number of nodes and OpenMP scheduling strategies. First, runs were conducted across 1, 2, and 4 nodes while keeping MPI ranks and threads constant. After identifying the 2-node configuration as the best among static runs, additional experiments were performed using dynamic scheduling with different chunk sizes. The results showed that static scheduling consistently outperformed dynamic scheduling, confirming that the best configuration was the 2-node static setup.